Apache Kafka Tutorial Series

From Basics to Production-Ready Patterns

Bash Scripts & Best Practices

About This Tutorial

- **Comprehensive Kafka Learning Path**
- **Bash-based Examples** - Easy to run and understand
- **Real-world Scenarios** - Production-ready patterns
- **Progressive Learning** - Basics to Advanced

Repository Structure:

- ✅ Basic Chapters (01-06): Foundation
- ✅ Advanced Chapters (A01-A06): Production patterns

Part 1: Basics

Getting Started with Kafka

Chapter 01: Environment Setup

What You'll Learn:

- ✅ Install Kafka locally or with Docker
- ✅ Start Zookeeper and Kafka brokers
- ✅ Verify connectivity
- ✅ Understanding Kafka components

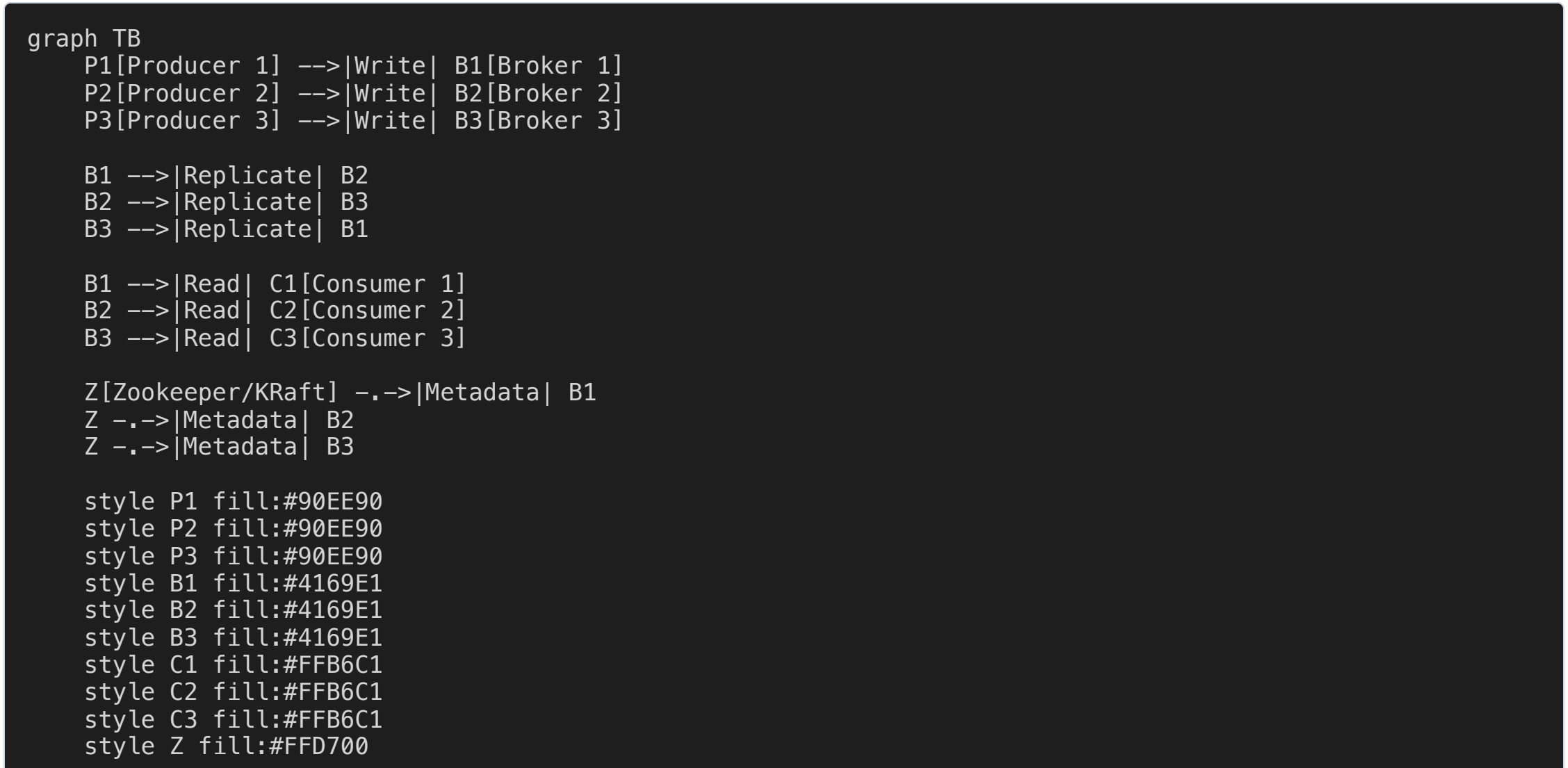
Key Scripts:

```
./bash/chapters/01-environment/check_connection.sh
./bash/chapters/01-environment/start_kafka.sh
```

Docker Setup:

```
make kafka-apache-start    # Pure Apache Kafka
```

Kafka Architecture Overview



Chapter 02: Topics

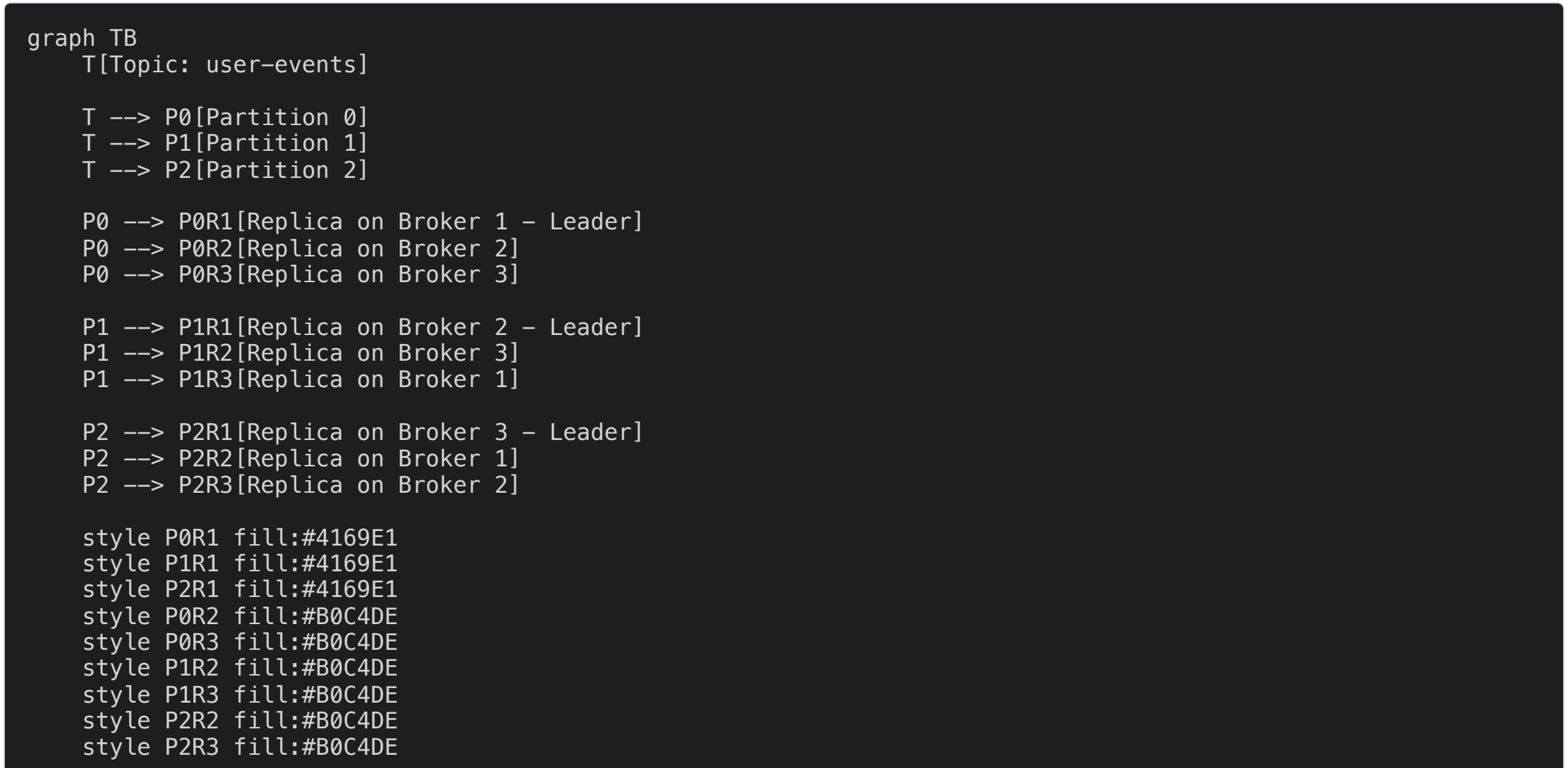
What You'll Learn:

- ✅ Create topics with custom configurations
- ✅ List and describe topics
- ✅ Understand partitions and replication
- ✅ Delete topics

Key Concepts:

- **Topic**: Category/feed name for messages
- **Partition**: Ordered, immutable sequence of records
- **Replication Factor**: Number of copies across brokers

Topic Architecture



Creating Topics

Basic Topic Creation:

```
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic user-events \
  --partitions 3 \
  --replication-factor 1
```

With Custom Configuration:

```
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic critical-events \
  --partitions 6 \
  --replication-factor 3 \
  --config retention.ms=604800000 \
  --config compression.type=lz4
```

Chapter 03: Producers

What You'll Learn:

- ✅ Send messages to topics
- ✅ Message keys and values
- ✅ Batch sending
- ✅ Console producer usage

```
sequenceDiagram
    participant App as Application
    participant P as Producer
    participant B1 as Broker (Leader)
    participant B2 as Broker (Follower)

    App->>P: Send message
    P->>P: Serialize
    P->>P: Partition selection
```

Sending Messages

Single Message:

```
echo "user-123:login event" | \
  kafka-console-producer.sh \
    --bootstrap-server localhost:9092 \
    --topic user-events \
    --property "parse.key=true" \
    --property "key.separator=:"
```

Batch Messages:

```
./bash/chapters/03-producer-console/send_batch_messages.sh

# Sends multiple messages efficiently:
# user-1:{"action":"login","timestamp":1234567890}
# user-2:{"action":"purchase","timestamp":1234567891}
# user-3:{"action":"logout","timestamp":1234567892}
```

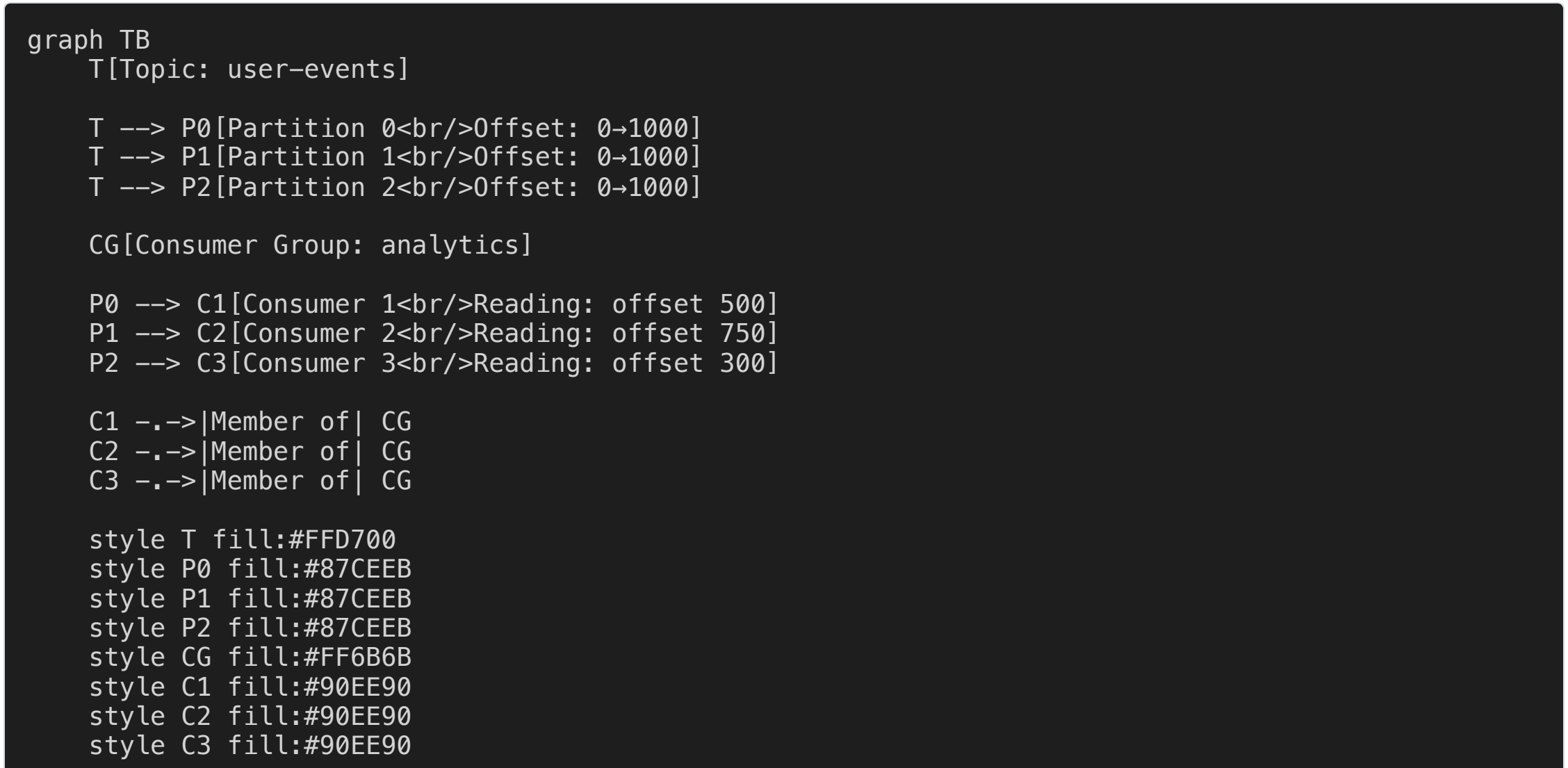
Chapter 04: Consumers

What You'll Learn:

- ✅ Read messages from topics
- ✅ Consumer groups and offset management
- ✅ Reading from beginning vs. latest
- ✅ Consuming with keys

```
sequenceDiagram
    participant App as Application
    participant C as Consumer
    participant B as Broker
    participant CG as Consumer Group Coordinator

    C->>CG: Join consumer group
    CG->>C: Assign partitions
```

Consumer Groups



Chapter 05: Consumer Groups

What You'll Learn:

- ✅ List consumer groups
- ✅ Describe group details
- ✅ View lag and offsets
- ✅ Reset offsets

List Groups:

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --list
```

Describe Group:

Chapter 06: Offsets

What You'll Learn:

- ✅ Understanding offset management
- ✅ Reset offsets to beginning/end
- ✅ Reset to specific offset or timestamp
- ✅ Replay messages

Reset to Beginning:

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group analytics \
  --topic user-events \
  --reset-offsets --to-earliest \
  --execute
```

Part 2: Advanced Patterns

Production-Ready Kafka

Advanced Chapter A01: Reliability

What You'll Learn:

- ✅ Idempotent producers
- ✅ ACK levels (`0`, `1`, `all`)
- ✅ Retries and timeouts
- ✅ Ordering guarantees

Key Configurations:

```
# Idempotent producer (exactly-once semantics)
enable.idempotence=true
acks=all
max.in.flight.requests.per.connection=5
retries=2147483647

# Request timeout
```

Delivery Guarantees Flow

```
sequenceDiagram
    participant P as Producer
    participant B as Broker
    participant C as Consumer

    Note over P,C: At-Most-Once (acks=0)
    P->>B: Send message
    Note over P: Don't wait for ACK
    P->>P: Consider sent ✓

    Note over P,C: At-Least-Once (acks=1)
    P->>B: Send message
    B->>P: ACK
    Note over P: Retry on timeout
    P-->>B: May send duplicate

    Note over P,C: Exactly-Once (idempotent)
    P->>B: Send (seq=1, producerId=X)
    B->>B: Check if seq=1 seen before
    alt First time
        B->>B: Store message
        B->>P: ACK
    else Duplicate
        B->>P: ACK (already stored)
    end
```

Delivery Guarantees Comparison

Guarantee	Config	Speed	Reliability	Use Case
At-Most-Once	`acks=0`	⚡⚡⚡ Fastest	⚠️ May lose	Metrics, logs
At-Least-Once	`acks=1`	⚡⚡ Fast	✅ Reliable	Most apps
Exactly-Once	`idempotence=true`	⚡ Good	✅✅ Strongest	Financial

Advanced Chapter A02: Performance

```
sequenceDiagram
    participant App as Application
    participant Mem as Memory Buffer
    participant Batch as Batch Builder
    participant Net as Network Thread
    participant B as Broker

    App->>Mem: send() - msg 1
    App->>Mem: send() - msg 2
    App->>Mem: send() - msg 3

    Note over Mem,Batch: Wait for batch.size OR linger.ms

    Mem->>Batch: Create batch (3 messages)
    Batch->>Batch: Compress (lz4/snappy/zstd)
    Batch->>Net: Ready to send
    Net->>B: Single network call
    B->>B: Decompress & write
    B->>Net: ACK
    Net->>App: Callbacks for all 3
```

Compression Comparison

Algorithm	Compression Ratio	CPU Usage	Speed	Best For
none	1:1	None	Fastest	Low latency
gzip	3-4:1	High	Slow	Max compression
snappy	2-2.5:1	Low	Fast	Balanced
lz4	2-2.5:1	Very Low	Very Fast	High throughput
zstd	3-3.5:1	Medium	Fast	Best overall

Benchmark Results (from our tests):

```
./bash/chapters/adv_chapter_02_performance/benchmark_compression.sh

none:   1000 msg/sec, 100 MB/sec, avg latency: 5ms
lz4:    950 msg/sec,  35 MB/sec, avg latency: 6ms
```

Advanced Chapter A03: Partitioning

What You'll Learn:

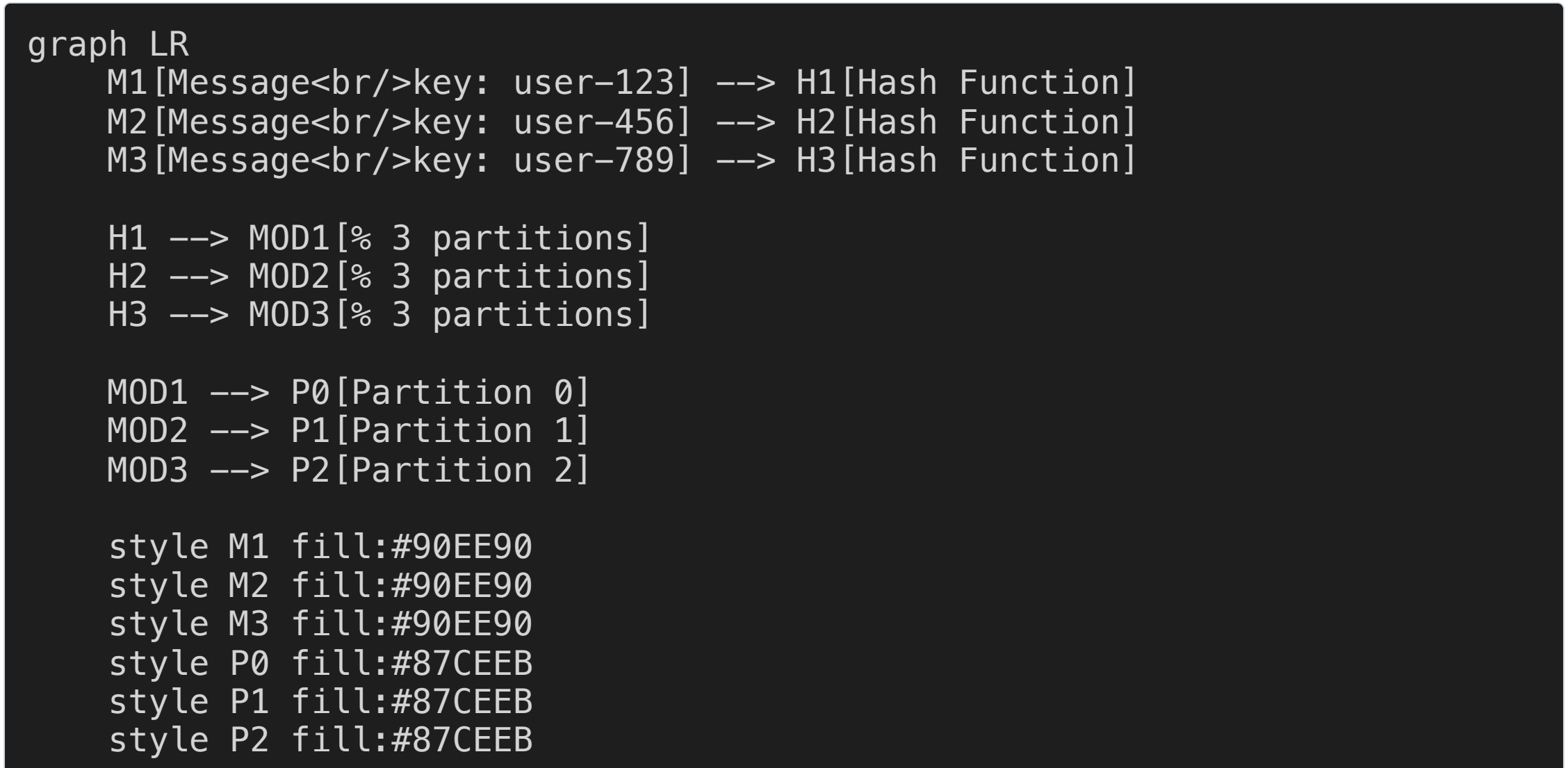
- ✅ Message keys and partition assignment
- ✅ Default partitioner (hash-based)
- ✅ Hot partition problems
- ✅ Custom partitioning strategies
- ✅ Ordering guarantees

Key Concepts:

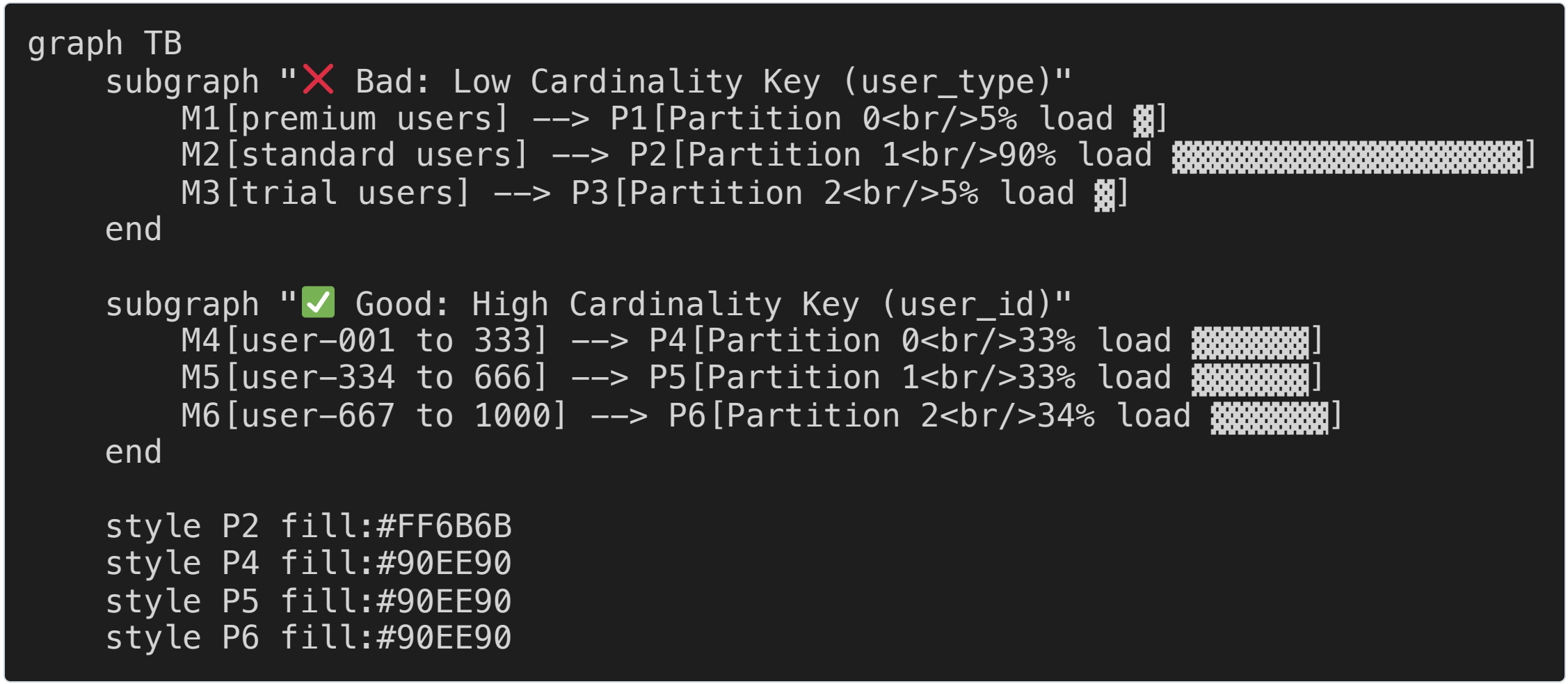
```
Message Key → Hash Function → Partition Assignment

Same Key → Same Partition → Guaranteed Order
```

Partition Selection:

Partitioning Strategy



Hot Partition Problem



Ordering Guarantees

```
sequenceDiagram
    participant P as Producer
    participant P0 as Partition 0
    participant P1 as Partition 1
    participant C as Consumer

    Note over P: Send messages
    P->>P0: user-123:event1 (offset 0)
    P->>P0: user-123:event2 (offset 1)
    P->>P0: user-123:event3 (offset 2)
    P->>P1: user-456:event4 (offset 0)
    P->>P1: user-456:event5 (offset 1)

    Note over C: Consume in order within partition
    P0->>C: event1, event2, event3 ✅ ORDERED
    P1->>C: event4, event5 ✅ ORDERED

    Note over C: But interleaving between partitions
    Note over C: Could receive: event1, event4, event2, event5, event3
```

Advanced Chapter A04: Serialization

```
sequenceDiagram
    participant App as Application
    participant Ser as Serializer
    participant SR as Schema Registry
    participant K as Kafka
    participant Des as Deserializer
    participant Con as Consumer

    Note over App,Con: Producer Side
    App->>Ser: Send User object
    Ser->>SR: Check/Register schema
    SR->>Ser: Return schema ID (123)
    Ser->>Ser: Encode: [0x00][ID:123][binary]
    Ser->>K: Send to Kafka

    Note over App,Con: Consumer Side
    K->>Des: Read message bytes
    Des->>Des: Extract schema ID (123)
    Des->>SR: Fetch schema for ID 123
    SR->>Des: Return schema definition
    Des->>Des: Decode binary using schema
    Des->>Con: Return User object
```

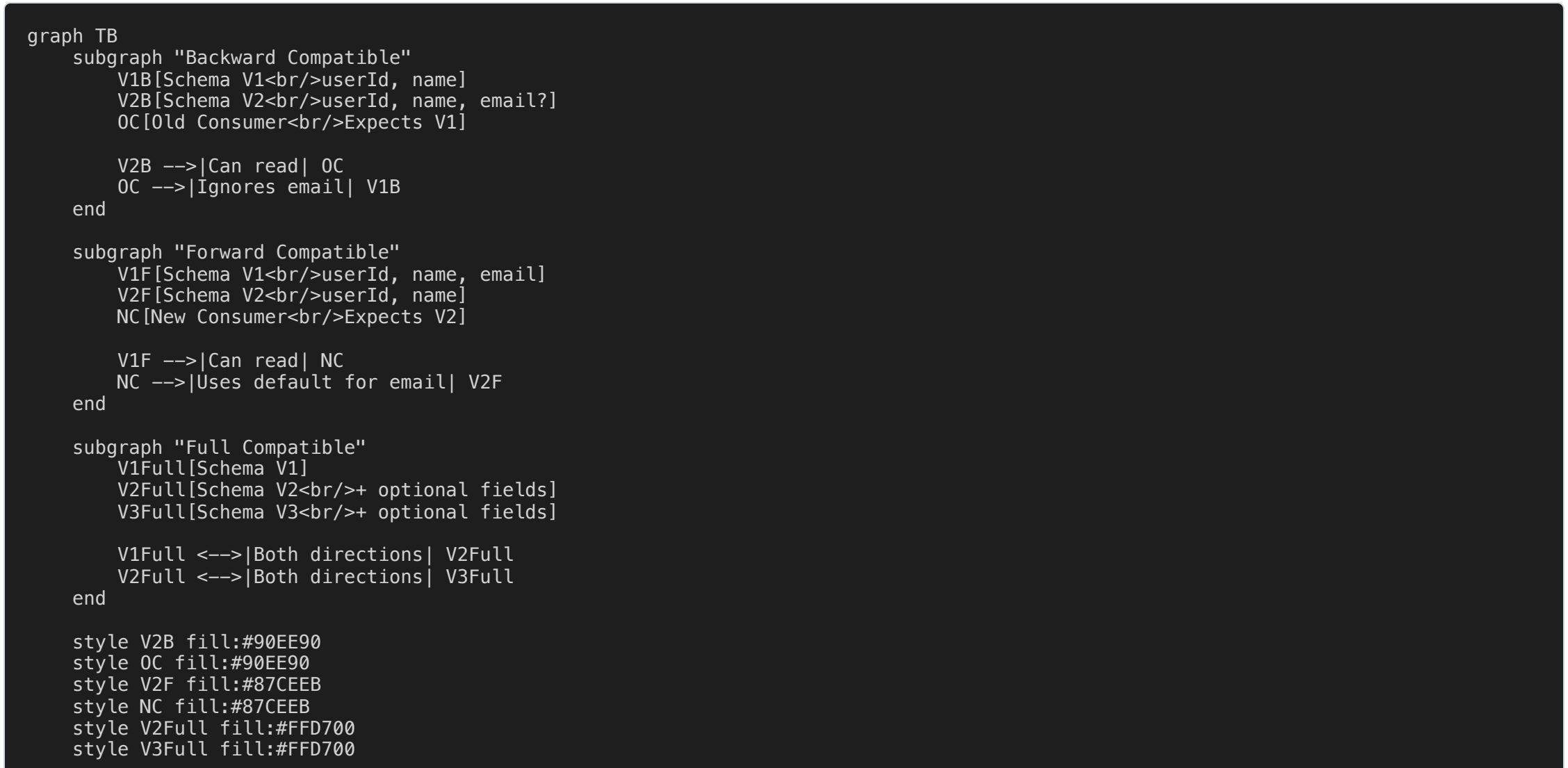
Serialization Formats Comparison

Format	Size	Speed	Schema	Evolution	Human Readable
String	Large	Fast	No	No	✅ Yes
JSON	Large	Medium	No	Manual	✅ Yes
Avro	Small	Fast	Yes	✅ Yes	❌ No
Protobuf	Small	Very Fast	Yes	✅ Yes	❌ No

Size Example (User record):

- String: ~100 bytes
- JSON: ~95 bytes
- Avro: ~38 bytes (60% smaller!)
- Protobuf: ~35 bytes

Schema Evolution



Advanced Chapter A05: Error Handling

What You'll Learn:

- ✅ Handle send failures (sync/async)
- ✅ Dead Letter Queue (DLQ) pattern
- ✅ Retry strategies
- ✅ Metrics and observability

Error Types:

Retriable (temporary):

- TimeoutException
- NetworkException
- NotEnoughReplicasException

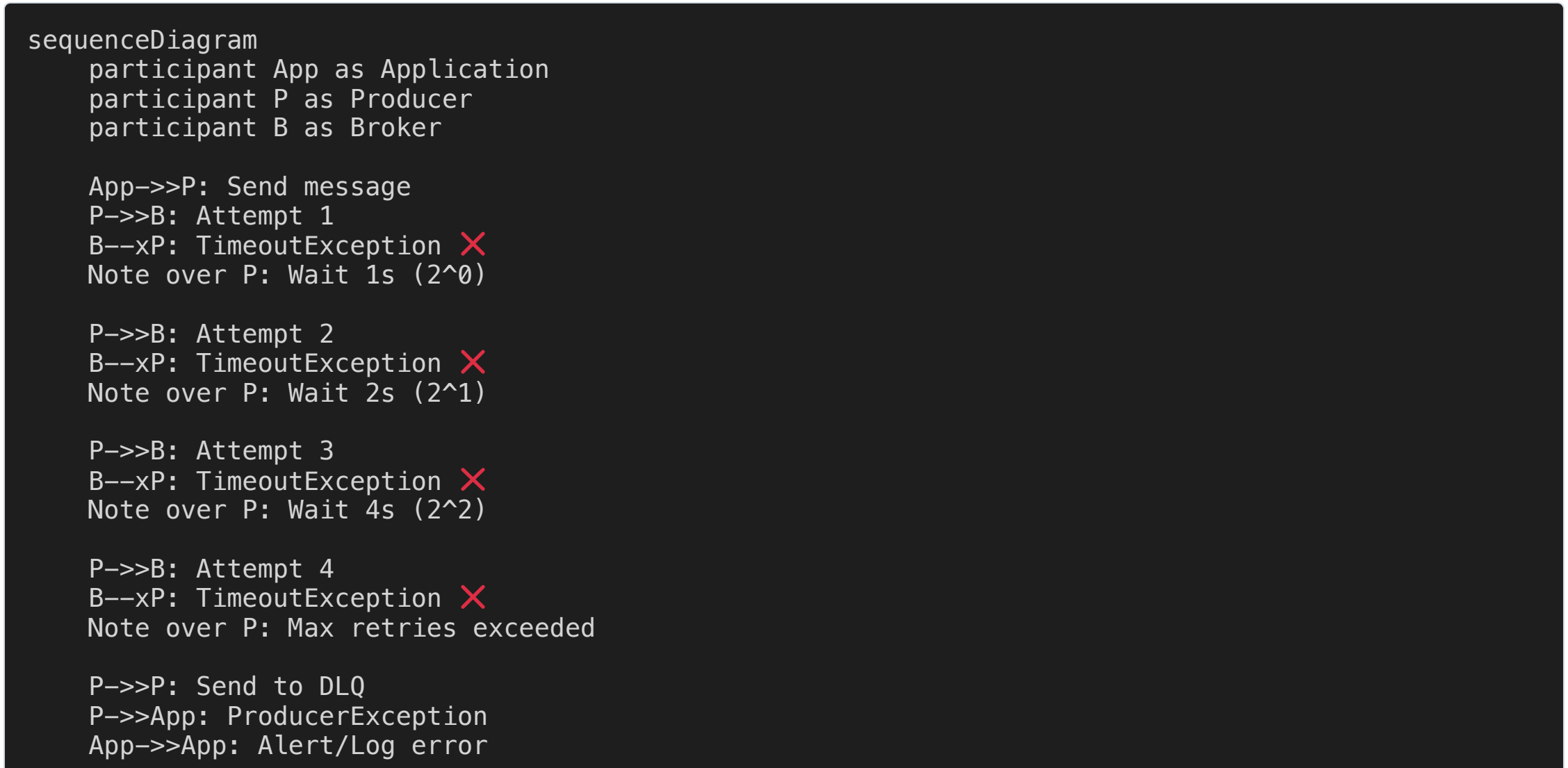
Fatal (permanent):

- RecordTooLargeException
- SerializationException
- AuthorizationException

Dead Letter Queue (DLQ) Pattern

```
sequenceDiagram
    participant P as Producer
    participant T as Topic: user-events
    participant C as Consumer
    participant DLQ as Topic: user-events.dlq
    participant M as Monitoring/Alerts

    Note over P,M: Normal Flow
    P->>T: Send message
    T->>C: Consume
    C->>C: Process successfully ✅

    Note over P,M: Error Flow - Retriable
    P->>T: Send message
    T->>C: Consume
    C->>C: Process fails (TimeoutException)
    C->>C: Retry #1, #2, #3

    Note over P,M: Error Flow - Fatal
    C->>C: Max retries exceeded ❌
    C->>DLQ: Send to DLQ with metadata
    DLQ->>M: Alert: Message in DLQ
    M->>M: Investigate & fix
    M->>T: Replay from DLQ (optional)
```

Retry Strategy with Exponential Backoff



Observability Metrics

Key Metrics to Monitor:

Metric	Target	Alert Threshold
Success Rate	>99.9%	<99%
Failure Rate	<0.1%	>1%
Average Latency	<50ms	>100ms
P99 Latency	<200ms	>500ms
Retry Count	Low	>10/min
DLQ Messages	0	>0
Buffer Usage	<80%	>90%

Monitoring Commands:

Advanced Chapter A06: Operations

What You'll Learn:

- ✅ Graceful shutdown
- ✅ Resource limits and protection
- ✅ Security (TLS/SASL)
- ✅ Production best practices

Graceful Shutdown:

```
# Trap signals
trap 'cleanup' SIGTERM SIGINT

cleanup() {
  echo "Flushing producer..."
  # Flush pending messages
  producer.flush()
```

Resource Limits & Protection

Producer Limits:

```
# Message size limit
max.request.size=1048576                    # 1 MB

# Buffer memory
buffer.memory=33554432                      # 32 MB

# Broker coordination
message.max.bytes=1048576                   # Must match producer

# Timeouts
max.block.ms=60000                          # Block for 60s max
request.timeout.ms=30000                    # Request timeout
delivery.timeout.ms=120000                  # Total delivery time
```

Why Limits Matter:

Security Best Practices

1. TLS Encryption:

```
security.protocol=SSL
ssl.truststore.location=/path/to/truststore.jks
ssl.truststore.password=${TRUSTSTORE_PASSWORD}
ssl.keystore.location=/path/to/keystore.jks
ssl.keystore.password=${KEYSTORE_PASSWORD}
```

2. SASL Authentication:

```
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
  required username="${KAFKA_USERNAME}" password="${KAFKA_PASSWORD}";
```

3. Credential Management:

Production Checklist

✅ Configuration:

- [ ] `enable.idempotence=true` (reliability)
- [ ] `acks=all` (durability)
- [ ] `compression.type=lz4` (efficiency)
- [ ] `batch.size` and `linger.ms` tuned (performance)
- [ ] Proper timeout values set
- [ ] Resource limits configured

✅ Monitoring:

- [ ] Producer/consumer metrics exposed
- [ ] Alerting on failures configured

Production Checklist (cont.)

✅ **Error Handling:**

- [ ] DLQ pattern implemented
- [ ] Retry logic with exponential backoff
- [ ] Fatal vs. retriable errors classified
- [ ] Circuit breakers in place
- [ ] Graceful degradation strategy

✅ **Operations:**

- [ ] Graceful shutdown hooks
- [ ] Health checks implemented
- [ ] Log aggregation configured
- [ ] Rocky-mode automation (CI/CD)

Performance Tuning Quick Reference

High Throughput:

```
batch.size=32768
linger.ms=20
compression.type=lz4
buffer.memory=67108864
acks=1
```

Low Latency:

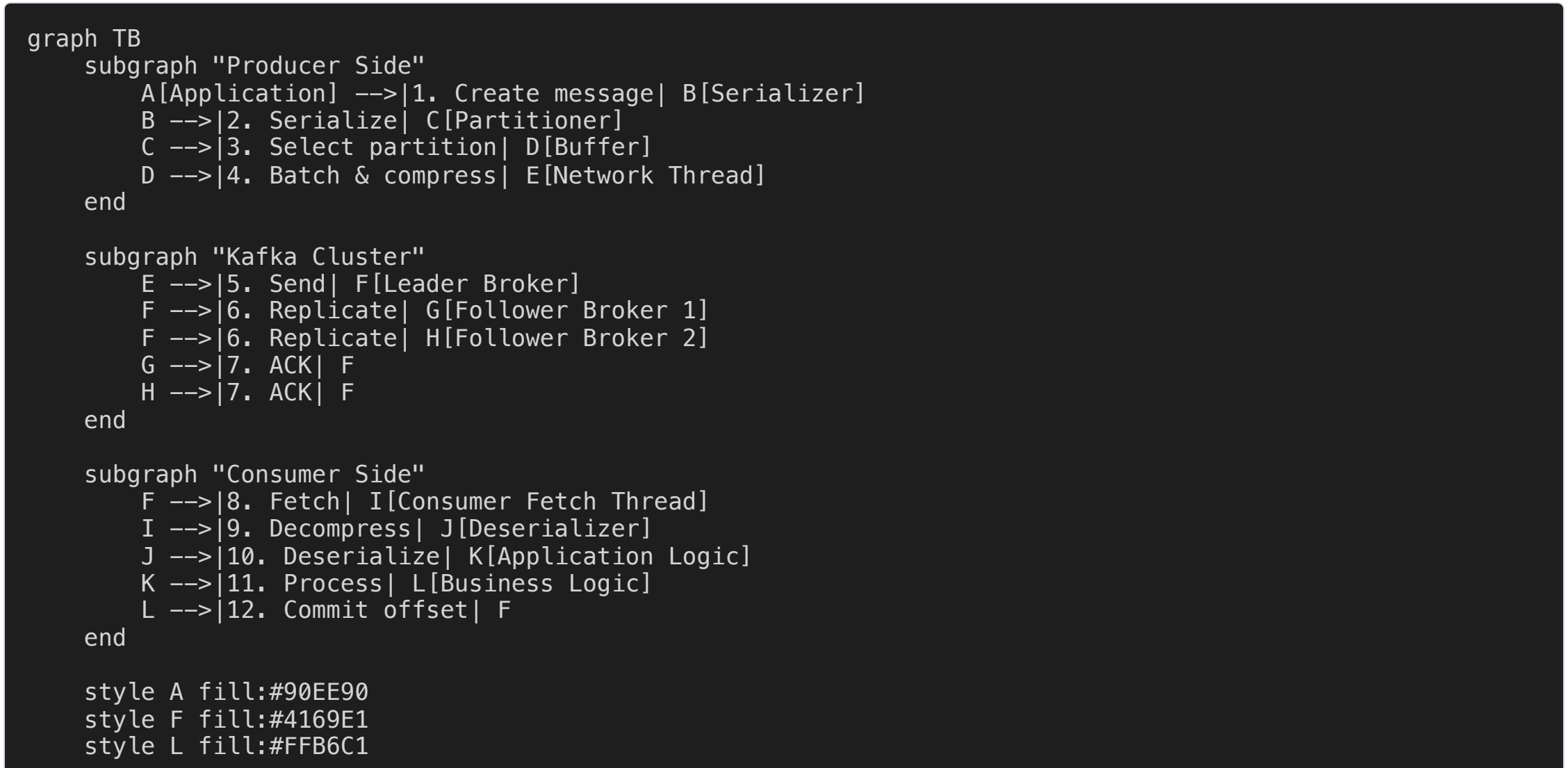
```
batch.size=16384
linger.ms=0
compression.type=none
acks=1
```

Maximum Reliability:

Common Pitfalls & Solutions

Problem	Symptom	Solution
Message Loss	Missing messages	`acks=all`, `enable.idempotence=true`
Duplicates	Same message twice	Enable idempotence
High Latency	Slow sends	Reduce `batch.size`, `linger.ms=0`
Low Throughput	Poor performance	Increase batch size, enable compression
Out of Order	Wrong sequence	`max.in.flight=1` or use keys
Consumer Lag	Falling behind	Add consumers, tune fetch settings
Hot Partitions	Uneven load	Better key selection, more partitions
Broker Overload	Timeouts	Scale brokers, reduce message rate

Complete Data Flow



Testing Your Setup

1. Connectivity Test:

```
./bash/chapters/01-environment/check_connection.sh
```

2. End-to-End Test:

```
# Start Kafka
make kafka-apache-start

# Create topic
kafka-topics.sh --create --topic test --partitions 3 ...

# Produce
echo "test-key:test-value" | kafka-console-producer.sh ...

# Consume
kafka-console-consumer.sh --topic test --from-beginning
```

Repository Quick Start

```
# 1. Clone repository
git clone https://github.com/your-repo/kafka-tutorials.git
cd kafka-tutorials

# 2. Start Kafka cluster
make kafka-apache-start

# 3. Set environment
source infra/env.local

# 4. Run basic examples
make test-ch01  # Environment check
make test-ch02  # Topics
make test-ch03  # Producer
make test-ch04  # Consumer

# 5. Run advanced examples
make test-adv-ch01  # Reliability
make test-adv-ch02  # Performance
make test-adv-ch03  # Partitioning
# ... and more
```

Learning Path Recommendation

Week 1: Foundations

- Day 1-2: Environment setup, basic concepts
- Day 3-4: Topics and producers
- Day 5: Consumers and consumer groups

Week 2: Advanced Patterns

- Day 1: Reliability and delivery guarantees
- Day 2: Performance tuning
- Day 3: Partitioning strategies
- Day 4: Serialization and schemas
- Day 5: Error handling

Resources & Next Steps

📚 Documentation:

- Official Kafka Docs: https://kafka.apache.org/documentation/
- Confluent Documentation: https://docs.confluent.io/
- Our README files in each chapter

🛠️ Tools:

- Kafka UI: http://localhost:8080 (included in Docker setup)
- Prometheus metrics: Available via JMX
- Grafana dashboards: Community templates

🎓 Further Learning:

- Kafka Streams for stream processing

Java Implementations

All Advanced Chapters Available in Java!

```
# Build Java projects
make java-reliability-build      # Chapter A01
make java-performance-build      # Chapter A02
make java-partitioning-build     # Chapter A03
make java-serialization-build    # Chapter A04
make java-errorhandling-build    # Chapter A05

# Run tests
make java-reliability-test
make java-performance-test
# ... and more
```

Features:

- ✅ Production-ready code

Demo Time! 🎯

Let's see it in action:

1. **Start Kafka Cluster**
2. **Create Topic with Partitions**
3. **Send Messages with Keys**
4. **Consume Messages**
5. **View Metrics and Lag**
6. **Simulate Failures**
7. **Show DLQ in Action**

```
# Follow along with the demo scripts
cd kafka-tutorials
make setup-and-test
```

Questions?

📧 **Contact:** [Your contact info]

🔗 **Repository:** [GitHub URL]

📖 **Documentation:** See README.md files

Thank You!

Happy Streaming with Kafka! 🚀

Remember:

- Start simple, iterate to production
- Monitor everything
- Test failure scenarios
- Read the docs

Kafka is powerful when done right!